

INFORME INTERNO DE INGENIERIA

Implementacion tecnica de DLSS 4.5 y DLAA

BioShock 1 Remastered en realidad virtual

Proyecto: BioShock VR DLSS/DLAA

Base tecnica analizada: version publica 0.2.11

Fecha: 10 de septiembre de 2026

Arquitectura, captura temporal, interoperabilidad D3D11/D3D12, ejecucion NGX
por ojo, sincronizacion y publicacion OpenXR.

Resumen ejecutivo

La integracion introduce una etapa temporal entre el renderizado D3D11 de BioShock y la entrega de cada ojo a OpenXR. El juego y el modulo VR permanecen en x86, mientras que NVIDIA NGX se ejecuta en dos procesos auxiliares x64: uno para el ojo izquierdo y otro para el derecho. Color, profundidad, movimiento y salida se intercambian como recursos GPU compartidos; el canal IPC solo transporta control y metadatos.

Resultado arquitectonico: dos cadenas temporales independientes y sincronizadas, sin copia de pixeles por CPU y sin reutilizar el historial del ojo contrario.

Alcance de nuestra contribucion

La base BioShock VR ya aportaba renderizado estereoscopico, camaras por ojo y publicacion OpenXR. Nuestra capa incorpora la captura de guias temporales, el puente entre arquitecturas, la evaluacion NGX, la coherencia de pareja y la recuperacion ante fallos.

- Captura del depth-stencil principal de la escena D3D11.
- Reconstruccion de motion vectors desde profundidad y camaras consecutivas.
- Recursos compartidos D3D11/D3D12 con fences y protocolo IPC version 8.
- Dos features NGX y dos historiales, completamente separados por ojo.
- Entrega del resultado del mismo frame a los swapchains OpenXR.

Diagrama general

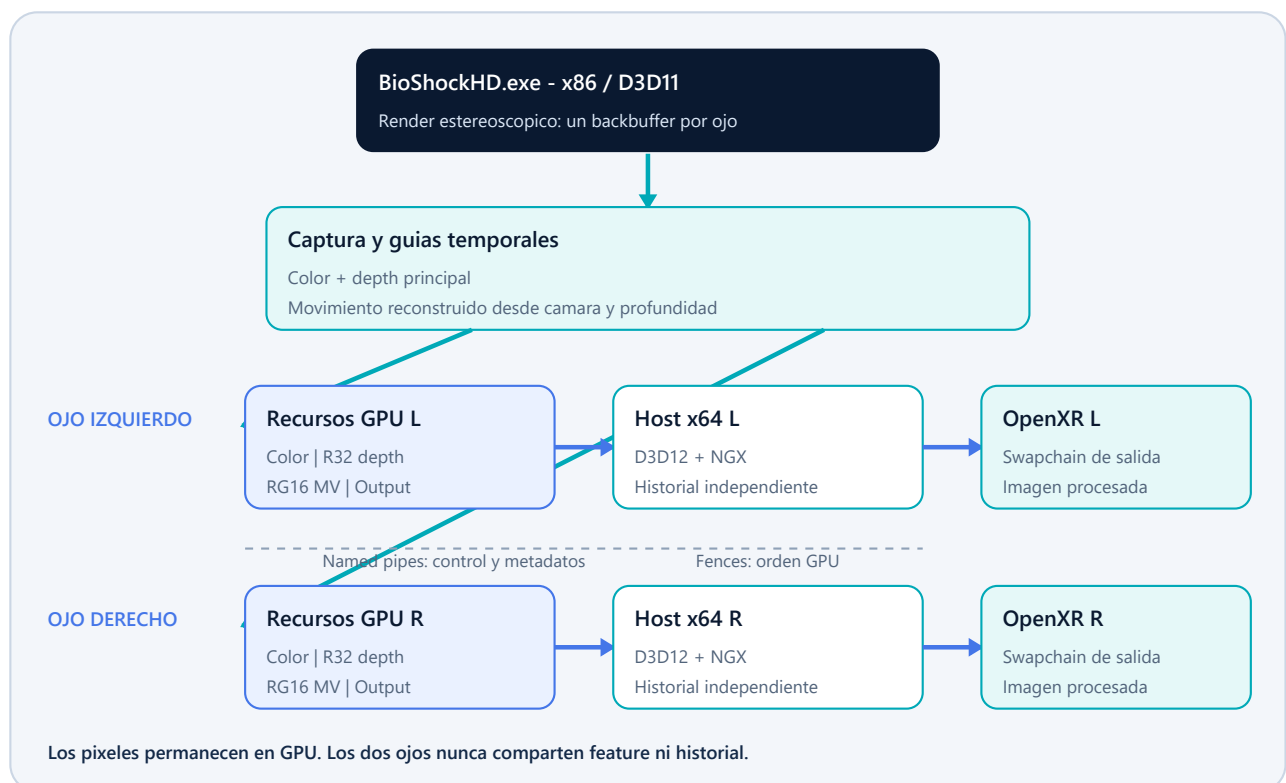


Figura 1. Ruta completa de datos y aislamiento estereoscopico.

1. Insercion en la canalizacion grafica

El punto de integracion se encuentra al final del render de cada ojo, antes de copiar el backbuffer a la imagen adquirida del swapchain OpenXR. En la ruta temporal se asocian el color, el depth principal, la camara, la proyeccion y un identificador monotono del render. Solo una combinacion coherente puede llegar a NGX.

Semantica de los modos

Modo	Entrada	Salida OpenXR	Feature NGX
DLAA	Resolucion nativa	Igual a entrada	Reconstruccion temporal y antialiasing 1:1
DLSS Super Resolution	Resolucion interna inferior	Resolucion superior	Reconstruccion temporal y escalado

DLAA exige que render y salida sean identicos. Super Resolution exige que la salida sea mayor en ambos ejes y conserve exactamente la relacion de aspecto. La pareja de swapchains OpenXR se crea a resolucion de salida, mientras que color, profundidad y movimiento conservan la resolucion interna.

Seleccion del preset temporal

El host consulta a NGX las resoluciones optimas y los rangos minimo/maximo de todos los niveles de calidad. Selecciona el candidato cuya resolucion optima sea mas cercana a la entrada solicitada y, a continuacion, verifica que esa entrada se encuentre realmente dentro del rango anunciado. Esto evita escoger un modo incorrecto cuando los rangos de varios presets se solapan.

- Modelo K para DLAA, Quality, Balanced y Ultra Quality.
- Modelo M para Performance.
- Modelo L para Ultra Performance.
- Los hints se fijan explicitamente para no heredar overrides externos.

2. Captura de profundidad

BioShock no expone una textura de profundidad preparada para un consumidor temporal. La implementacion observa los DSV, clears, cambios de estado y operaciones que pueden escribir profundidad. Entre un conjunto acotado de candidatos selecciona la superficie principal de escena y descarta recursos con formato, dimensiones, mip, array o MSAA incompatibles.

La superficie elegida se copia a una textura R24G8_TYPELESS. Una vista R24_UNORM_X8_TYPELESS realiza la conversion normalizada de 24 bits al leerla. Un compute shader escribe el valor hardware sin alterar en una salida R32_FLOAT para NGX.

Seguridad D3D11: si el DSV sigue enlazado al cerrar el ojo, se guarda el estado completo del Output Merger, se desconecta temporalmente el depth, se realiza la copia y se restaura el estado. Asi se evita un hazard de lectura/escritura.

La optimizacion de reutilizacion solo acepta una copia anterior cuando pertenece a la misma textura y vista y no ha existido ninguna escritura potencial desde entonces. Los seriales de escritura cubren draws, clears y operaciones no rastreadas.

3. Generacion de motion vectors

El motor no produce un buffer de movimiento utilizable. Se genera mediante un compute shader D3D11, separadamente para cada ojo. La reconstruccion utiliza profundidad, posicion y base de la camara actual, FOV observado y la camara anterior del mismo ojo.

1	Leer profundidad Se obtiene el valor hardware del pixel y se normaliza segun la convencion D3D.
2	Reconstruir posicion Con near/far y las tangentes del FOV se recupera la posicion tridimensional en mundo.
3	Reproyectar La posicion se transforma a la camara anterior del mismo ojo.
4	Calcular movimiento Se escribe <code>previousPixel - currentPixel</code> en R16G16_FLOAT.

```
motionVector = previousPixel - currentPixel
```

Los vectores se expresan en pixeles de render con escala (1,1). Si no hay historial valido, si la geometria queda detras de la camara anterior o si cambia la proyeccion, el shader escribe movimiento cero y la llamada temporal marca Reset.

Validacion de la historia

- La camara debe haberse publicado exactamente una vez para ese render.
- El identificador de camara debe coincidir con el de la proyeccion.
- La proyeccion observada y la declarada deben coincidir dentro de tolerancia.
- El historial de un ojo avanza de dos en dos porque los renders globales alternan izquierda y derecha.
- Un salto, teletransporte o cambio de FOV invalida la reproyeccion anterior.

4. Puente GPU entre x86 y x64

El modulo x86 no carga NGX directamente. Crea cuatro recursos D3D11 compartibles por ojo y entrega sus handles NT a un host x64. El host duplica los handles desde el proceso del juego y abre las mismas asignaciones como recursos D3D12. Los datos de imagen permanecen residentes en GPU durante todo el recorrido.

Slot	Formato / tamaño	Funcion
Color	Formato tipado, resolucion de render	Backbuffer del ojo
Depth	R32_FLOAT, resolucion de render	Profundidad temporal
Motion	R16G16_FLOAT, resolucion de render	Desplazamiento en pixeles
Output	Resolucion de salida	Resultado NGX

Cada ojo dispone ademas de un named pipe propio y dos fences compartidos. El protocolo IPC version 8 transporta dimensiones, formatos, handles, convencion de profundidad, escala de movimiento, reset, jitter e identificador de frame. Nunca transporta el contenido de las texturas.

5. Feature NGX por ojo

Se lanzan dos procesos x64 aislados. Cada uno crea su dispositivo y cola D3D12, su feature NGX, sus fences y su historial. Esta separacion es obligatoria en VR: compartir la historia entre ojos mezclaria proyecciones y profundidades incompatibles.

Parametro	Valor aplicado
Color / Output	Texturas compartidas del ojo
Depth	R32_FLOAT con profundidad hardware
Motion vectors	R16G16_FLOAT, baja resolucion, escala 1:1
Reset	Primer frame o historia invalidada
Jitter	(0,0) en la implementacion actual
Exposure	AutoExposure; pre-exposure y escala = 1
Depth convention	Normal y finita para BioShock

El jitter permanece en cero porque la proyeccion rasterizada del motor no se desplaza. Enviar un jitter que no existio en la imagen romperia la correspondencia entre color, profundidad y guias.

6. Sincronizacion del mismo frame



Figura 2. Contrato de sincronizacion para un ojo y un frame N.

- El juego copia Color, Depth y Motion a los recursos compartidos.
- D3D11 señala fenceIn(N), hace Flush y envia el mensaje de frame.
- El host introduce una espera GPU, evalua NGX y señala fenceOut(N).
- El cliente espera de forma acotada observando tambien el proceso host.
- D3D11 inserta la espera GPU final y copia Output al swapchain OpenXR.

No se consume la salida de un frame anterior. Para reducir la latencia visible, el ojo izquierdo se envia mientras BioShock continua construyendo el derecho; parte del trabajo auxiliar de OpenXR tambien se solapa con la evaluacion del segundo host.

7. Coherencia estereoscopica

Los renders reciben identificadores monotonos. Una pareja valida exige que el derecho sea exactamente izquierdo + 1; la historia individual de un ojo avanza + 2. Un ojo derecho sin hermano izquierdo, una etiqueta fuera de orden o una guia incoherente impiden usar la ruta temporal para esa pareja.

Condicion	Respuesta
Pareja coherente	DLAA o DLSS SR en ambos ojos
Profundidad/camara ausente	Se invalida ese frame y se solicita Reset
Right sin Left o IDs no consecutivos	Se rechaza la pareja temporal completa
Fallo tras enviar Left	Se restaura la copia retenida cuando aun no se ha publicado
Fallo de host o timeout	Se libera la espera, se desactiva el puente y se protege la sesion

Cuando una pareja pierde coherencia se invalidan ambos historiales y se fuerza un limite limpio antes de reanudar la reconstruccion temporal. Esto evita contaminar un ojo con datos temporalmente incompatibles del otro.

8. Recuperacion y ausencia de bloqueos

La sincronizacion incluye escapes tanto en CPU como en GPU. Antes de registrar la dependencia D3D11 sobre la salida, el cliente espera con timeout y observa si el proceso host sigue vivo. Si la evaluacion falla, el host se aliza la fence de salida desde CPU para que el juego nunca quede esperando indefinidamente.

- Un protocolo incompatible se rechaza durante el handshake, antes de interpretar estructuras con distinto tama o.
- Los recursos recibidos se verifican contra dimensiones, formatos y flags declarados.
- La perdida del dispositivo o del pipe invalida la sesion temporal.
- Los dos procesos auxiliares pertenecen a un Job Object y terminan con el juego.
- Los cambios de resolucion reconstruyen recursos y features, sin reutilizar historia antigua.

9. Limitaciones tecnicas actuales

Movimiento de objetos

Los vectores proceden de la camara. Personajes, manos y armas no disponen de movimiento individual y pueden generar ghosting local.

Jitter

Se envia cero hasta que el mismo desplazamiento pueda aplicarse a la matriz rasterizada del motor.

Proyeccion

Near, far, FOV y convencion de profundidad deben coincidir con la proyeccion real del frame.

Frames no estereoscopicos

Menus, cargas y cinematicas sin pareja de camaras valida no alimentan el historial temporal.

Validacion visual

Las pruebas automaticas verifican contratos y recuperacion; la calidad binocular final requiere comprobacion en visor fisico.

10. Resultado de la implementacion

La solucion convierte una aplicacion D3D11 x86 sin motion vectors nativos en un productor temporal compatible con NGX x64, manteniendo aislamiento completo entre ojos y entregando el resultado directamente a OpenXR.

El diseño resuelve cuatro problemas principales: incompatibilidad de arquitectura, ausencia de guias temporales, sincronizacion entre APIs y coherencia estereoscopica. El resultado no es un filtro aplicado sobre la imagen final, sino una integracion temporal dentro del ciclo de renderizado y presentacion VR.

Mapa del codigo

Componente	Responsabilidad
src/game/bioshock1r/temporal_guides.cpp	Captura de profundidad, camaras y generacion de movimiento
src/core/gfx/dlss45_client.cpp	Recursos D3D11, procesos, IPC, fences, submit y resolve
src/core/vr/openxr_runtime.cpp	Activacion, parejas estereo, swapchains y publicacion
components/dlss-host/src/feed_ipc.h	Contrato binario del puente version 8
components/dlss-host/host/	Host x64 D3D12 y evaluacion NVIDIA NGX

Documento tecnico generado a partir de la arquitectura y el codigo fuente del proyecto BioShock VR DLSS/DLAA.